arXiv:2302.12173v2 [cs.CR] 5 May 2023

Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection

Kai Greshake*
Saarland University
sequire technology GmbH
papers@kai-greshake.de

Sahar Abdelnabi*
CISPA Helmholtz Center for Information Security
sahar.abdelnabi@cispa.de

Shailesh Mishra
Saarland University
shmi00001@uni-saarland.de

Christoph Endres
sequire technology GmbH
christop.endres@sequire.de

Thorsten Holz
CISPA Helmholtz Center for Information Security
holz@cispa.de

Mario Fritz
CISPA Helmholtz Center for Information Security
fritz@cispa.de

ABSTRACT

Large Language Models (LLMs) are increasingly being integrated into various applications. The functionalities of recent LLMs can be flexibly modulated via natural language prompts. This renders them susceptible to targeted adversarial prompting, e.g., Prompt Injection (PI) attacks enable attackers to override original instructions and employed controls. So far, it was assumed that the user is directly prompting the LLM. But, what if it is *not* the user prompting? We argue that *LLM-Integrated Applications* blur the line between data and instructions. We reveal new attack vectors, using *Indirect Prompt Injection*, that enable adversaries to remotely (without a direct interface) exploit LLM-integrated applications by strategically injecting prompts into data likely to be retrieved. We derive a comprehensive taxonomy from a computer security perspective to systematically investigate impacts and vulnerabilities, including data theft, worming, information ecosystem contamination, and other novel security risks. We demonstrate our attacks' practical viability against both real-world systems, such as Bing's GPT-4 powered Chat and code-completion engines, and synthetic applications built on GPT-4. We show how processing retrieved prompts can act as arbitrary code execution, manipulate the application's functionality, and control how and if other APIs are called. Despite the increasing integration and reliance on LLMs, effective mitigations of these emerging threats are currently lacking. By raising awareness of these vulnerabilities and providing key insights into their implications, we aim to promote the safe and responsible deployment of these powerful models and the development of robust defenses that protect users and systems from potential attacks.

KEYWORDS

Large Language Models, Indirect Prompt Injection

1 INTRODUCTION

Foundation and instruction-following [63] Large Language Models (LLMs) [43, 62] are changing our lives on many levels, not only for researchers and practitioners but also for the general public. Shortly after its release, ChatGPT [1] gained immense popularity, attracting over 100 million users in a short period of time [10]. Furthermore, there is a constant stream of new models, including the more-advanced GPT-4 [62] and smaller white-box models [68, 68].

*Contributed equally.

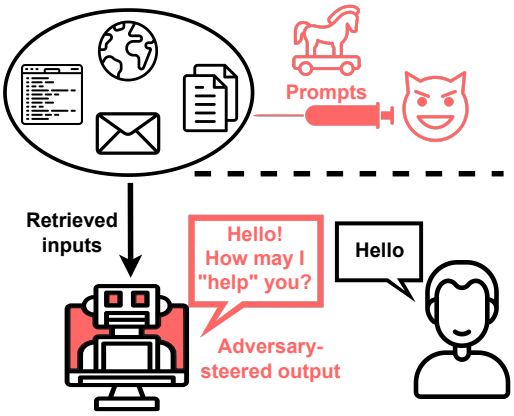

Figure 1: With LLM-integrated applications, adversaries could control the LLM, without direct access, by *indirectly* injecting it with prompts placed within sources retrieved at inference time.

LLM-Integrated Applications. Beyond their impressive capabilities, LLMs are now integrated into other applications at a widespread fast-paced rate. Such tools can offer interactive chat and summary of the retrieved search results or documents and perform actions on behalf of the user by calling other APIs [9]. In the few months after ChatGPT, we witnessed Bing Chat [31], Bard [4], Microsoft 365 and Security Copilots [19, 20], and numerous ChatGPT plugins [9]– with new announcements on almost a daily basis. However, we argue that this AI-integration race is not accompanied by adequate guardrails and safety evaluations.

Prompt Injection. Attacks against ML models typically involve powerful algorithms and optimization techniques [35]. However, the easily extensible nature of LLMs' functionalities via natural prompts can enable more straightforward attack tactics. Even under black-box settings with mitigation already in place [53], malicious users can exploit the model through *Prompt Injection* (PI) attacks that circumvent content restrictions or gain access to the model's original instructions [18, 33, 67].

Indirect Prompt Injection. Augmenting LLMs with retrieval blurs the line between *data* and *instructions*. Adversarial prompting has been so far assumed to be performed directly by a malicious user exploiting the system. In contrast, we show that adversaries

can now *remotely affect other users' systems* by strategically injecting the prompts into data likely to be retrieved at inference time. If retrieved and ingested, these prompts can *indirectly* control the model (see Figure 1). Recent incidents already show that retrieved data can accidentally elicit unwanted behaviors (e.g., hostility) [25]. In this work, we take this idea further and investigate what an adversary can purposefully do to modify the behavior of LLMs in applications, potentially affecting millions of benign users. Given the unprecedented nature of this attack vector, there are numerous new approaches to delivering such attacks and the myriad of threats they can cause. To address this unexplored challenge, we first develop a systematic taxonomy that examines these emerging vulnerabilities from a computer security perspective.

Impact. We show that *Indirect Prompt Injection* can lead to full compromise of the model at inference time analogous to traditional security principles. This can entail remote control of the model, persistent compromise, theft of data, and denial of service. Furthermore, advanced AI systems add new layers of threat: Their capabilities to adapt to minimal instructions and autonomously advance the attacker's goals make them a potent tool for adversaries to achieve, e.g., disinformation dissemination and user manipulation. In this paper, we construct the first examples of such attacks.

In summary, our main **contributions** are:

- We introduce the concept of Indirect Prompt Injection (IPI) to compromise LLM-integrated applications—a completely uninvestigated attack vector in which retrieved prompts themselves can act as "arbitrary code".
- We develop the first taxonomy and systematic analysis of the threat landscape associated with IPI in LLM-integrated applications.
- We showcase the practical feasibility of these attacks on both real-world and synthetic systems, emphasizing the need for robust defenses.
- We share all our demonstrations on our GitHub repository[1] and all developed attack prompts in the Appendix of this paper to foster future research and contribute to building an open framework for the security assessment of LLM-integrated applications.

2 PRELIMINARIES AND RELATED WORK

We review preliminaries and recent work on LLMs, prompt injection, and similar security aspects of LLMs.

Prompting to Augment LLMs with APIs. One way to augment LLMs with tools is to instruct them on how to use these tools. Toolformer [71] shows that it is possible to train LLMs to output API calls by inferring which API to call, when, and which arguments to pass. This is done in a self-supervised manner by first leveraging in-context learning to generate potential training data for fine-tuning. Similarly, ReAct [79] is another method to equip LLMs with tools using Chain-of-Thought prompting [77]. It generates both task-specific actions and verbal reasoning, given intermediate observations from the environment.

From LLMs to Autonomous Agents. Moving beyond merely integrating LLMs into applications, there are ongoing active research and engineering efforts to create more autonomous systems [40, 56, 72] (i.e., given a task, the foundation model itself, as a central system, outlines the solution and potentially connects to other APIs or models to achieve the sub-tasks). Auto-GPT [3] is another open-source project that is advertised as able to self-prompt to complete a task. Moreover, Park et al. [64] recently designed an interactive simulation environment in which "AI agents" interact and autonomously plan tasks (e.g., throwing a party). Agents were equipped with external memory streams that store observations and a retrieval mechanism to recall relevant memories. The framework utilizes prompting to instruct agents to extract high-level "reflections" and recursively create and update plans. In our work, we also observe that LLMs might be able to execute attackers' goals given only high-level descriptions. As we move towards more autonomy and less human oversight, it is crucial to study the security implications of such systems.

LLM Safety. LLMs might make up facts ("hallucinate"), generate polarized content, or reproduce biases, hate speech, or stereotypes [39, 41, 47, 57, 61, 65, 78]. This partially stems from pretraining on massive crawled datasets. One of the motivations for leveraging *Reinforcement Learning from Human Feedback* (RLHF) [63, 75] is to better align LLMs with human values and avert these unwanted behaviors [37]. OpenAI reports that GPT-4 shows less tendency, although still possible, to hallucinate or generate harmful content [62]. However, it continues to reinforce social biases and worldviews, and it might also have other emergent risks, such as social engineering and risks associated with interactions with other systems [62]. Unwanted behaviors are already manifesting in LLM-integrated applications. Shortly after its launch, Bing Chat raised public concerns over unsettling outputs [12, 25], urging Microsoft to limit the chatbot's conversations with users [24]. Search-augmented chatbots can also make factual mistakes [17, 26], blur the boundary between trusted and untrusted sources [16], and cite each other in an unprecedented incident of automated misinformation Ping-Pong [16]. These incidents occurred without any adversarial prompting; the risks can be further exacerbated by such.

Adversarial Prompting and Jailbreaking. Perez et al. [67] showed that current models, such as GPT-3 and applications built on it, are vulnerable to prompt injection (PI). They design prompts that either *hijack* the original goal of the model or *leak* the original prompts and instructions of the application. Shortly after ChatGPT's release, many users reported that filtering can be circumvented by a prompting technique called "jailbreaking" [18, 21]. This typically involves drawing a hypothetical scenario in which the bot has no restrictions, or simulating a "developer mode" that can access the uncensored model's output. OpenAI reports that jailbreaking is harder for GPT-4 but still possible [62], as also shown in our work.

LLMs as Computers. Zhou et al. [81] recently highlighted that LLMs can conceptually be analogous to black-box computers that execute programs coded via natural language instructions. Kang et al. [53] further synergized LLMs with classical computer security to derive methods such as program obfuscation, payload splitting, and virtualization to bypass current APIs' filters. We build on these

[1] https://github.com/greshake/llm-security

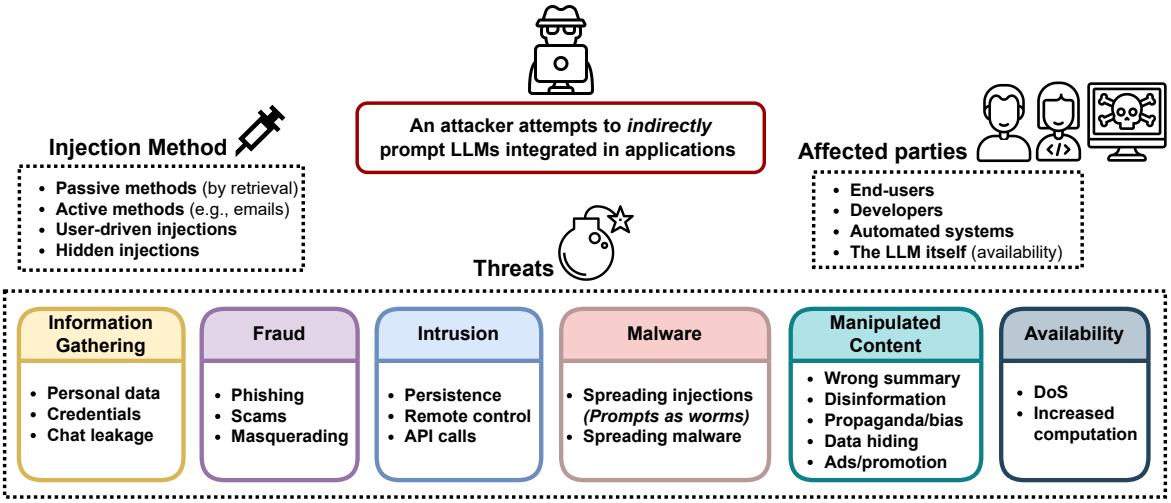

Figure 2: A high-level overview of new indirect prompt injection threats to LLM-integrated applications, how the prompts can be injected, and who can be targeted by these attacks.

observations and point out another critical insight; when augmenting LLMs with retrieval, *processing* untrusted retrieved data would be analogous to *executing* arbitrary code, and the line between *data* and *code* (i.e., instructions in natural language) would get *blurry*.

Other Adversarial ML Attacks. PI can be categorized under the general umbrella of the empirical evaluation of ML security and robustness [35]. Moreover, *indirect PI*, as introduced in our work, is conceptually similar to previous work on backdoor attacks against language models [36] or hijacking the functionality of models [69]. Unlike these attacks, PI requires less technical skills, ML capabilities, cost to run the attack, and almost no control over models and knowledge about them. This could give attackers economic and practical incentives to exploit such vulnerabilities and position them within an essential territory that the ML security research community might have ignored so far [35].

3 ATTACK SURFACE OF LLM-INTEGRATED APPLICATIONS

Prompt injection (PI) attacks pose a significant threat to the security of LLMs. While PI attacks have been primarily limited to individuals attacking their own LLM instances (or a public model such as ChatGPT [18]), integrating LLMs with other applications might make them susceptible to untrusted data ingestion where malicious prompts have been placed. We call this new threat *indirect prompt injections* and demonstrate how such injections could be used to deliver targeted payloads. As illustrated in Figure 3, this technique might allow attackers to gain control of LLMs by crossing crucial security boundaries with a single search query.

Recent LLMs may behave like computers executing programs [51, 53, 81]. Thus, we draw insights from the classical computer security domain to design a new set of attack techniques. We provide a high-level overview of the threat model in Figure 2, covering the possible injection delivery methods, the different threats, and the possible affected individuals or systems.

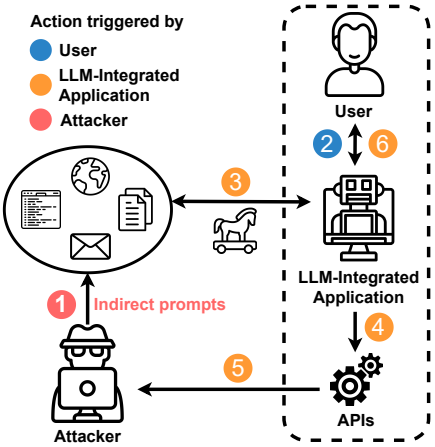

Figure 3: Attackers can plant instructions ❶ that are retrieved ❸ when the user prompts ❷ the model. If the model has access to APIs and tools ❹, they can be used to communicate back to the attacker ❺ or perform unwanted actions. The compromised LLM might also influence the user directly ❻.

> **Key Message#1:** Retrieval unlocks new doors for prompt injections where current (likely brittle) input filtering techniques are not applied in some of today's systems.

3.1 Injection Methods

There are potentially several ways to deliver the injection of malicious prompts, depending on the application itself. We broadly outline them below.

Passive Methods. These methods rely on retrieval to deliver injections. For example, for search engines, the prompts could be placed within public sources (e.g., a website or social media posts) that would get retrieved by a search query. Attackers could use